

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 1 · Fundamentos de Java y control de flujo

Introducción a Java: JVM, bytecode, tipos primitivos y referencias

Variables, operadores, conversiones y promociones

Estructuras de control: if / else-if / switch, bucles while, do-while, for

Entrada/salida por consola (Scanner)

Métodos: paso de parámetros por valor y por referencia

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 1
Valoración	Teoría y práctica sobre Java básico

Parte 1: Opción múltiple y Verdadero/Falso

5 puntos — 0,5 ca

Dado el código siguiente, ¿qué se imprime?

```
int a = 7;
double b = 2.0;
System.out.println(a / b);
```

- a) 3
- b) 3.0
- c) 3.5
- d) Error de compilación

RESPUESTA / SOLUCIÓN

c) 3.5 — Al dividir un int por un double, Java promociona el int a double y el resultado es 3.5.

¿Cuál de estos bucles garantiza que el cuerpo se ejecute al menos una vez?

- a) while (cond) { }
- b) for (int i=0; i<n; i++) { }
- c) do { } while (cond);
- d) Ninguno lo garantiza

RESPUESTA / SOLUCIÓN

c) do-while — Evalúa la condición al final, por lo que ejecuta el cuerpo al menos una vez.

Si ejecutamos números con tipo byte, ¿qué operación causaría un error de compilación por posible pérdida de precisión?

- a) byte r = (byte) (a + b);
- b) byte r = a + b;
- c) int r = a + b;
- d) long r = a + b;

RESPUESTA / SOLUCIÓN

b) byte r = a + b; — a + b se promociona a int al operar, por lo que asignarlo a byte requiere un cast explícito.

En Java, cuando se pasa un array a un método, las modificaciones que haga el método sobre los elementos se reflejan en el array original.

RESPUESTA / SOLUCIÓN

Verdadero — En Java todo se pasa por valor, pero las referencias a objetos (incluidos los arrays) copian la dirección. Al modificar elementos a través de esa referencia, se modifica el objeto original.

Dado el switch, ¿qué valor se imprime si op = 2?

```
int op = 2;
switch (op) {
    case 1: System.out.print("A"); break;
    case 2: System.out.print("B");
    case 3: System.out.print("C"); break;
    default: System.out.print("D");
}
```

- a) B
- b) BC
- c) BCD
- d) A

RESPUESTA / SOLUCIÓN

b) BC — El case 2 no tiene break, por lo que 'cae' (fall-through) hacia el case 3 y también imprime C antes de llegar al break.

En Java, todas las variables de tipo primitivo se almacenan en el montículo (heap) cuando se declaran dentro de un método.

RESPUESTA / SOLUCIÓN

Falso — Las variables locales de tipo primitivo se almacenan en la pila (stack), no en el heap.

¿Qué imprime el siguiente bucle?

```
int i = 0;
while (i < 5) {
    if (i % 2 == 0) System.out.print(i);
    i++;
}
```

- a) 024
- b) 135
- c) 01234
- d) 1234

RESPUESTA / SOLUCIÓN

a) 024 — Imprime los valores de i que son pares (0, 2, 4).

¿Qué método de Scanner permite leer un número entero correctamente?

- a) sc.next()
- b) sc.nextInt()
- c) sc.nextDouble()
- d) sc.nextLine()

RESPUESTA / SOLUCIÓN

b) sc.nextInt() — Lee y devuelve el siguiente token como int.

Dado el código:

```
int x = 10;
int z = (x > 5) ? x * 2 : x / 2;
System.out.println(z);
```

- a) 10
- b) 5
- c) 20
- d) 2

RESPUESTA / SOLUCIÓN

c) 20 — El operador ternario evalúa la condición (true), por lo que $z = 10 * 2 = 20$.

¿Cuál de estas declaraciones de un método es el correcto para un método que no devuelve nada y no recibe parámetros?

- a) `public static int main()`
- b) `public static void ejemplo()`
- c) `public static String ejemplo()`
- d) `public static example()`

RESPUESTA / SOLUCIÓN

- b) `public static void ejemplo()` — `void` indica que no devuelve nada.

Parte 2: Análisis y depuración

3 puntos

El siguiente programa tiene 3 errores. Encuéntralos y explica por qué fallan. (1,5 pts)

```
public class Promedio {
    public static void main(String[] args) {
        int[] notas = {4, 7, 3, 9};
        double media = calcular(notas[]);
        System.out.println("Media: " + media);
    }
    static double calcular(int[] n) {
        int suma = 0;
        for (int i = 0; i <= n.length; i++) {
            suma += n[i];
        }
        return suma;
    }
}
```

RESPUESTA / SOLUCIÓN

1. `calcular(notas[])` → no se pasan los corchetes al invocar: debe ser calcular(notas).`
2. `i <= n.length` → desbordamiento: el último índice válido es length-1. Debe ser i < n.length.`
3. `return suma;` → no devuelve la media: debe ser `return suma / (double) n.length;` para que el resultado sea 7,25 (doble división real y no entera).

Analiza el siguiente código y determina qué imprime (traza manual). (1,5 pts)

```
public class Traza {
    public static void main(String[] args) {
        int a = 3;
        int b = 5;
        int c = metodo(a, b);
        System.out.println(a + " " + b + " " + c);
    }
    static int metodo(int x, int y) {
        x = x + 2;
        y = y + 1;
        return x * y;
    }
}
```

RESPUESTA / SOLUCIÓN

En Java los primitivos se pasan por valor: dentro de `metodo`, `x` se convierte en 5 e `y` en 6, y devuelve $5 \cdot 6 = 30$.

Sin embargo, `a` y `b` en `main` NO cambian porque el paso es por valor (se copian).

Salida: ``3 5 30``

Parte 3: Ejercicios de programación

7 puntos

Escribe un programa que pida un número entero positivo n y calcule la suma de todos los múltiplos de 3 o de 5 menores que n. (2 pts)

RESPUESTA / SOLUCIÓN

```
import java.util.Scanner;

public class Multiplos {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("n: ");

        int n = sc.nextInt();

        int suma = 0;

        for (int i = 1; i < n; i++)

            if (i % 3 == 0 || i % 5 == 0) suma += i;

        System.out.println("Suma: " + suma);

    }

}
```

Implementa un método que reciba un número entero y devuelva true si es primo y false en caso contrario. Úsalo en un programa que muestre los primeros 10 números primos. (2 pts)

RESPUESTA / SOLUCIÓN

```
public class Primos {

    static boolean esPrimo(int n) {

        if (n < 2) return false;

        for (int i = 2; i <= Math.sqrt(n); i++)

            if (n % i == 0) return false;

        return true;

    }

    public static void main(String[] args) {

        int encontrados = 0, n = 2;

        while (encontrados < 10) {

            if (esPrimo(n)) { System.out.println(n); encontrados++; }

            n++;

        }

    }

}
```

Escribe un programa que lea dos números por teclado y muestre un menú (sumar, restar, multiplicar, dividir). Usa un bucle do-while que se repita mientras el usuario no elija salir, y gestiona la división entre cero. (2 pts)

RESPUESTA / SOLUCIÓN

```
import java.util.Scanner;

public class Calculadora {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int opcion;

        do {

            System.out.println("1.Sumar 2.Restar 3.Multiplicar 4.Dividir 0.Salir");

            opcion = sc.nextInt();

            if (opcion == 0) break;

            System.out.print("a y b: ");

            double a = sc.nextDouble(), b = sc.nextDouble();

            switch (opcion) {

                case 1: System.out.println("Suma: " + (a + b)); break;

                case 2: System.out.println("Resta: " + (a - b)); break;

                case 3: System.out.println("Prod: " + (a * b)); break;

                case 4: if (b == 0) System.out.println("División entre cero");

                        else System.out.println("Div: " + (a / b)); break;

                default: System.out.println("Opción no válida");

            }

        } while (opcion != 0);

    }

}
```

Explica con tus palabras la diferencia entre pasar un parámetro por valor y por referencia en Java, y entre tipos primitivos y de referencia. Pon un ejemplo en el que se aprecie la diferencia. (1 pto)

RESPUESTA / SOLUCIÓN

Java SIEMPRE pasa parámetros por valor: se copia el valor de la variable o la referencia del objeto. Para primitivos, las modificaciones dentro del método no afectan a la variable original. Para objetos/arrays, se copia la referencia, de modo que modificar el contenido (p.ej. `notas[i]=5`) sí afecta al original, pero reasignar la variable (``obj = new ...``) no lo afecta fuera.

Parte 4: Pregunta teórica

1 punto

Explica la diferencia entre los bucles ``while``, ``do-while`` y ``for``. ¿Cuándo conviene usar cada uno? Razonar la respuesta.

RESPUESTA / SOLUCIÓN

``while``: evalúa la condición al inicio → 0 o más iteraciones. Útil cuando no sabemos cuántas veces iteraremos y solo conocemos la condición.

``do-while``: evalúa al final → 1 o más iteraciones. Útil cuando el cuerpo debe ejecutarse como mínimo una vez (p. ej., menús).

``for``: compacto para iteraciones con contador conocido (índice + condición + incremento). Útil para recorrer arrays o rangos determinados.

Plantilla de respuestas

(Páginas en blanco)